

Universidad del Valle de Guatemala

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Construcción de Compiladores

Proyecto 2.1

Explorando otras aplicaciones

Intérprete SQL

con almacenamiento en archivos

ANTLR 4.13.2 • C++17 • AST • IDE web

Repositorio: rama Compiladores2-Proyecto3

Catedrático

Ing. Pablo Koch

Integrantes

Luis Gonzalez — 23353

Joel Jaquez — 23369

Guatemala, 31 de octubre de 2026

Versión del código documentada: `037ac10`

Índice

1 Introducción **1**

2 Objetivos **1**

2.1 Objetivo general 1

2.2 Objetivos específicos 1

3 Correspondencia con el enunciado **2**

4 Arquitectura y reutilización **2**

4.1 Flujo de procesamiento 2

4.2 Qué se conserva y qué cambia respecto a Compiscript 3

4.3 Organización del núcleo 4

5 Reconocimiento del lenguaje y construcción del AST **4**

5.1 Gramática SQL 4

5.2 Precedencia y árbol sintáctico concreto 5

5.3 AST y decoración semántica 5

6 Análisis semántico y manejo de errores **5**

6.1 Catálogo y ámbitos de consulta 5

6.2 Tipos, conversiones y restricciones 6

6.3 Códigos y ubicación de los diagnósticos 6

7 Diseño del almacenamiento en archivos **7**

7.1 Distribución de los archivos 7

7.2 Catálogo: esquemas, tipos y restricciones 7

7.3 Registros CSV y valores especiales 8

7.4 Posiciones de fila y lectura directa 8

7.5 Confirmación, recuperación y concurrencia 8

8 Interpretación y ejecución de consultas **9**

8.1 DDL, INSERT, UPDATE y DELETE 9

8.2 Orden de evaluación de SELECT 9

8.3 JOIN, agrupación y agregaciones 9

8.4 Subconsultas y lógica de NULL 10

9 Índices y planificación del acceso **10**

10 Interfaz gráfica y evidencia de funcionamiento **12**

10.1 Consulta filtrada y plan de ejecución 12

10.2 Tokens 13
10.3 Árbol sintáctico concreto 14
10.4 AST visible y tipos resueltos 15
10.5 JOIN, agregaciones y HAVING 16
10.6 Subconsultas con IN 17
10.7 EXISTS correlacionado y subconsulta escalar 18
10.8 Catálogo e índices persistidos 19
10.9 Rechazo de entradas inválidas 20

11 Pruebas y resultados de validación **21**
11.1 Metodología 21
11.2 Regresiones incorporadas 21
11.3 Evidencia reproducible 22

12 Trazabilidad del desarrollo **23**

13 Compilación y ejecución **23**

14 Alcance, límites y mejoras futuras **24**

15 Conclusiones **25**

Índice de figuras

1 SELECT con WHERE, ORDER BY, resultados y acceso por índice de edad. . . 12
2 Tokens de la consulta SELECT nombre FROM estudiantes WHERE edad >= 18. 13
3 Árbol sintáctico concreto de una consulta, en modo de análisis. 14
4 AST decorado con la proyección, la tabla y el predicado de comparación. 15
5 INNER JOIN con COUNT, MAX, GROUP BY, HAVING y ORDER BY. 16
6 IN con SELECT anidado y acceso al índice de estudiantes. 17
7 EXISTS correlacionado con una referencia a la fila externa. 18
8 Explorador del almacenamiento con el contenido real de un índice .idx. 19
9 Errores semánticos con código, sentencia y ubicación en el editor. 20
10 Error de ejecución por división entre cero. 20
11 Reporte de la batería del intérprete SQL: 117 aprobadas y cero fallos reales. . . . 22

1 Introducción

Este informe presenta la implementación del **intérprete SQL con almacenamiento en archivos** correspondiente al Proyecto 2.1, *Explorando otras aplicaciones*. En el repositorio, esta entrega se identifica mediante la rama `Compiladores2-Proyecto3`, por ser la tercera aplicación desarrollada en la serie de proyectos de compiladores.

El objetivo es aplicar análisis léxico, análisis sintáctico, construcción de un árbol de sintaxis abstracta (AST), análisis semántico e interpretación a un lenguaje de consultas. Una sentencia válida se ejecuta sobre esquemas, registros e índices administrados por el propio programa. El sistema **no utiliza SQLite, MySQL, PostgreSQL ni otro DBMS** para almacenar los datos o ejecutar SQL.

El núcleo está escrito en **C++17**. ANTLR genera el lexer y el parser desde `SQL.g4`; un Visitor construye el AST; el analizador semántico resuelve tablas, columnas y tipos; y el intérprete produce resultados o modifica archivos de texto. Flask sirve como adaptador entre el ejecutable nativo y el IDE web. La interfaz permite inspeccionar tanto el resultado como las fases que lo producen.

La infraestructura se deriva de los proyectos de Compiscript, pero esta entrega **interpreta el AST directamente**: el flujo SQL no genera TAC, temporales ni registros de activación. Se reutilizan las herramientas de análisis y presentación, mientras que el lenguaje, el catálogo y la ejecución de consultas son componentes propios de SQL.

La validación actual comprende **117 pruebas de integración aprobadas**: 76 casos clasificados como válidos y 41 errores esperados. Además de los requisitos funcionales, se verifican Unicode, precisión numérica, reapertura de archivos, acceso directo por índices y recuperación frente a fallos de escritura.

2 Objetivos

2.1 Objetivo general

Desarrollar un sistema que interprete y ejecute el subconjunto SQL requerido por el enunciado sobre un mecanismo de almacenamiento propio, utilizando las etapas de análisis de un compilador para validar las sentencias antes de operar sobre archivos.

2.2 Objetivos específicos

- Definir una gramática SQL y generar el lexer y el parser con ANTLR.
- Construir un AST independiente del árbol sintáctico concreto y decorarlo con tipos y referencias de columnas.
- Validar tablas, columnas, tipos, índices, restricciones y subconsultas mediante diagnósticos con ubicación.
- Implementar DDL, DML, consultas, JOIN, agregaciones y agrupación.
- Persistir esquemas, tipos, registros e índices en archivos de texto y mantenerlos consistentes ante modificaciones y fallos.
- Usar índices en los predicados compatibles y leer directamente las filas candidatas del CSV.

- Integrar el pipeline en una interfaz que muestre tokens, árbol, AST, errores, resultados y archivos de la base de datos.
- Aportar pruebas reproducibles y evidencia visual de funcionamiento.

3 Correspondencia con el enunciado

El enunciado organiza la entrega en ocho incrementos. La tabla siguiente relaciona cada uno con los componentes implementados y su forma de verificación. Los ejemplos ejecutables se encuentran en `backend/examples/sql/`.

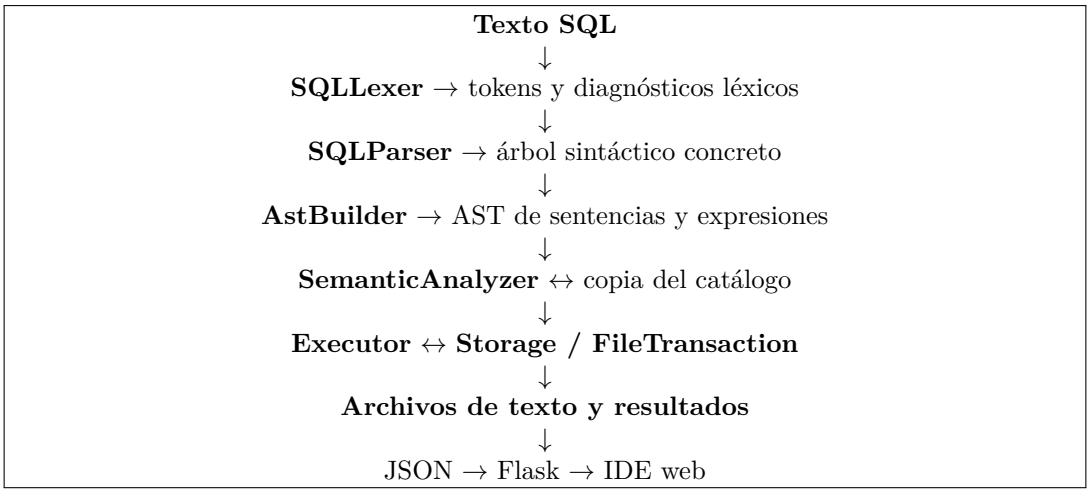
Entrega	Implementación	Evidencia
1. Lexer y parser	`SQL.g4`, código ANTLR, clasificación de tokens y listeners propios.	Tokens y árbol; errores `LEX001`/`SYN001`.
2. AST	`AstBuilder`, nodos de sentencias y expresiones, serialización independiente.	Pestaña AST y pruebas de precedencia.
3. Semántica	Catálogo, ámbitos de consulta, tipos, restricciones y diagnósticos `SEM001`–`SEM020`.	Rechazo previo y ubicación de errores.
4. CREATE, DROP, INSERT	Creación y eliminación de tablas; filas de `VALUES`; catálogo persistente.	Ejemplos 01 y 02; CSV y metadatos.
5. SELECT y DML	Proyección, FROM, WHERE, AND/OR, ORDER BY, UPDATE y DELETE.	Ejemplos 03 y 04; resultados y archivos.
6. JOIN y agregaciones	INNER JOIN, COUNT, SUM, AVG, MIN, MAX, GROUP BY y HAVING.	Ejemplo 05; verificaciones por grupo.
7. Índices y subconsultas	CREATE/DROP INDEX; mantenimiento DML; IN/NOT IN, EXISTS/NOT EXISTS y subconsultas escalares.	Ejemplo 06; planes de acceso y reapertura.
8. Integración	CLI JSON, adaptador Flask, IDE y batería de pruebas.	Interfaz ejecutable y 117 casos.

Tabla 1: Trazabilidad de los incrementos solicitados.

4 Arquitectura y reutilización

4.1 Flujo de procesamiento

La función pública `runScript(source, options)` coordina las fases. `RunOptions` selecciona el archivo, la carpeta de almacenamiento y el modo de ejecución o de análisis. El mismo servicio atiende al CLI, las pruebas y el IDE.



ANTLR proporciona los mecanismos para reconocer el lenguaje y recorrer sus árboles generados; las reglas de significado y ejecución pertenecen a la aplicación [2, 3].

Si existe un error léxico o sintáctico, se omiten la semántica y la ejecución. Si hay errores semánticos en cualquier sentencia del script, no se ejecuta ninguna. En modo análisis se construyen y validan las estructuras, pero no se ejecutan sentencias SQL.

Entrada	**AST**	**Semántica**	**Ejecución**
Error léxico o sintáctico	omitido	omitida	omitida
Error semántico	disponible	rechazada	omitida
Script válido, modo análisis	decorado	correcta	omitida
Script válido, modo ejecución	decorado	correcta	habilitada

Tabla 2: Separación y omisión de fases del pipeline SQL.

Un error que depende de los datos se reporta durante la interpretación. En ese caso se detiene el script: las sentencias anteriores ya confirmadas permanecen guardadas y las siguientes se omiten. Esta política se distingue de los errores semánticos, que se detectan antes de comenzar a ejecutar.

4.2 Qué se conserva y qué cambia respecto a Compiscript

Componente	**Reutilización**	**Adaptación SQL**
ANTLR y CMake	Runtime C++ y generación de código.	Nueva gramática y nuevas clases generadas.
Diagnósticos	Listener que recoge errores y conserva ubicación.	Mensajes en español y códigos de las fases SQL.
Árbol y JSON	Serialización de nodos y escape de cadenas.	AST, catálogo, resultados, planes y archivos.
Servicio y CLI	Un núcleo independiente con salida JSON.	`-db`, `-analyze`, `-catalog` y `-reset`.
Flask e IDE	Editor, archivos, pestañas, visor SVG y reporte de pruebas.	Resultados SQL y explorador del almacenamiento.
Semántica y ejecución	Separación entre reconocimiento y significado.	Catálogo relacional, consultas, índices y persistencia propios.

Tabla 3: Reutilización de infraestructura y componentes específicos de SQL.

El módulo anterior permanece en `backend/compiscript/`. Su compilación es opcional con `BUILD_COMPISCRIPT=ON`; el proyecto predeterminado compila el intérprete SQL. La existencia del código anterior en el repositorio no lo incorpora al pipeline de esta entrega.

4.3 Organización del núcleo

Archivo o grupo	Responsabilidad
`grammar/SQL.g4`	Definición del lexer y el parser.
`generated/SQL*.cpp`	Reconocimiento generado por ANTLR.
`ast.h`, `ast_builder.cpp`	Modelo del AST y Visitor que lo construye.
`ast_serializer.cpp`	AST decorado en JSON para el visor del IDE.
`semantic_analyzer.cpp`	Resolución de nombres, tipos y restricciones estáticas.
`catalog.h/.cpp`	Esquemas, columnas, restricciones e índices.
`value.h/.cpp`	Valores, comparación, conversión, fechas y Unicode.
`executor.h/.cpp`	Interpretación de DDL, DML, consultas y expresiones.
`storage.h/.cpp`	CSV, índices, posiciones, caché y bloqueo de archivos.
`file_transaction.h/.cpp`	Preparación, respaldos, confirmación y recuperación.
`service.cpp`, `main.cpp`	Orquestación y opciones del CLI.
`json.cpp`, listeners y serializers	Contrato JSON, diagnósticos, tokens y árbol concreto.

Tabla 4: Componentes de `backend/sql/`; cabeceras en `include/sql/` y fuentes en `src/`.

5 Reconocimiento del lenguaje y construcción del AST

5.1 Gramática SQL

La gramática combinada `SQL.g4` admite secuencias de sentencias separadas por punto y coma. Sus alternativas principales corresponden a `CREATE TABLE`, `DROP TABLE`, `CREATE INDEX`, `DROP INDEX`, `INSERT`, `UPDATE`, `DELETE` y `SELECT`. Se permiten punto y coma final opcional, sentencias vacías y comentarios de línea o de bloque.

La opción `caseInsensitive=true` hace que palabras reservadas e identificadores se reconozcan sin distinguir mayúsculas. El AST normaliza los nombres a minúsculas; el contenido de las cadenas conserva su texto. Los tokens reservados tienen prefijo `K_`, por ejemplo `K_SELECT` y `K_NULL`, para evitar conflictos con macros de C/C++ y de Windows.

Las reglas distinguen enteros, decimales, cadenas entre comillas simples, booleanos, NULL y literales DATE. En una cadena, la comilla simple se escapa duplicándola: `'O"Brien'`. Los comentarios `-` y `/* ... */` se descartan durante el análisis léxico.

5.2 Precedencia y árbol sintáctico concreto

Las alternativas de expresión establecen la precedencia de signos unarios, multiplicación y división, suma y resta, comparaciones, pruebas de NULL, pertenencia, NOT, AND y OR. Los paréntesis cambian la agrupación. Así, `2 + 3 * 4` produce 14 y `TRUE OR FALSE AND FALSE` produce TRUE.

El parser construye el *parse tree*, que conserva las reglas intermedias y los tokens de puntuación. `tree_serializer` lo convierte en nodos con nombre, texto, categoría, línea, columna e hijos. Este árbol permite explicar cómo se reconoció una sentencia, pero no es la estructura que interpreta directamente el ejecutor.

5.3 AST y decoración semántica

`AstBuilder` deriva de `SQLBaseVisitor`. Cada alternativa crea nodos con significado y descarta comas, punto y coma y paréntesis de agrupación. Las sentencias se almacenan en un `std::variant`; las expresiones se distinguen por `ExprKind`.

Tipo de expresión	**Ejemplos**
Literal y columna	`18`, `'Ana'`, `TRUE`, `e.nombre`.
Unaria y binaria	`-edad`, `NOT activo`, `edad >= 18`.
Pruebas y pertenencia	`IS NULL`, `IN (...)`, `NOT IN (SELECT ...)`.
Subconsulta	EXISTS y subconsulta escalar.
Agregación	COUNT, SUM, AVG, MIN y MAX, con DISTINCT cuando corresponde.

Tabla 5: Familias de expresiones representadas en el AST.

Para `SELECT nombre FROM estudiantes WHERE edad >= 18`, el AST contiene una proyección, una tabla y un predicado de comparación. La semántica agrega el tipo de cada expresión y, para las columnas, una posición `slot` en la fila combinada y una profundidad `depth`. El ejecutor consulta estas posiciones en lugar de volver a resolver cada nombre por texto.

Los nodos conservan la ubicación y el fragmento fuente. Por ello el AST puede servir tanto para ejecutar como para presentar diagnósticos y explicar las decisiones en el IDE.

6 Análisis semántico y manejo de errores

6.1 Catálogo y ámbitos de consulta

El catálogo sustituye a la tabla de símbolos de un lenguaje de programación general: describe tablas, columnas, tipos, restricciones e índices. `SemanticAnalyzer` trabaja con una copia del catálogo y procesa el script en orden. Un CREATE válido incorpora su esquema a esa copia, de modo que las sentencias siguientes pueden referirse a la tabla sin que todavía se haya escrito ningún archivo.

Cada SELECT abre un ámbito con las columnas de FROM y de los JOIN ya incorporados. El predicado ON solo puede usar las tablas disponibles en ese punto. Un nombre sin calificador que coincide con varias columnas produce una referencia ambigua. Los alias distinguen las tablas y sus usos.

En una subconsulta se abre otro ámbito. La búsqueda comienza en la consulta interior y continúa hacia los niveles externos. Si una columna se resuelve en un nivel exterior, se registra `depth >`

`0` y la subconsulta se marca como correlacionada.

6.2 Tipos, conversiones y restricciones

Tipo	Representación y validación
INT	Entero con signo de 64 bits; conversiones y operaciones comprobadas frente a desbordamientos.
FLOAT	Punto flotante `double`; admite promoción desde INT y persistencia decimal que reconstruye el mismo valor binario.
VARCHAR(n)	Texto UTF-8 con máximo de n puntos de código Unicode. Se preservan comillas, comas y saltos de línea.
BOOLEAN	TRUE o FALSE; los operadores lógicos exigen operandos compatibles.
DATE	Formato YYYY-MM-DD; comprueba mes, día y años bisiestos. Un literal de texto válido puede convertirse al asignar o comparar con DATE.

Tabla 6: Tipos mínimos solicitados y decisiones de implementación.

NULL representa ausencia de valor y no constituye un sexto tipo de columna. Las comparaciones ordinarias con NULL producen un resultado desconocido; WHERE solo conserva filas cuyo predicado resulta TRUE. Se implementan AND y OR con lógica de tres valores y cortocircuito.

Las restricciones adicionales incluyen PRIMARY KEY, NOT NULL, UNIQUE y FOREIGN KEY. Las referencias deben apuntar a columnas PRIMARY KEY o UNIQUE de tipo compatible. Se emplea la política RESTRICT para impedir borrar o cambiar una clave que otra tabla referencia.

Las restricciones que dependen de valores almacenados se comprueban en ejecución. Por ejemplo, una columna puede ser válida en el esquema y, sin embargo, un INSERT fallar porque repite una clave o referencia un registro inexistente.

6.3 Códigos y ubicación de los diagnósticos

Los diagnósticos contienen código, categoría, severidad, mensaje, número de sentencia, línea y columna. Las posiciones se muestran al usuario desde 1. El IDE permite seleccionar un error para ir al punto correspondiente del editor. Los listeners propios sustituyen a los mensajes de consola de ANTLR y producen `LEX001` o `SYN001`.

Código	Validación semántica
SEM001–004	Tabla inexistente; columna inexistente; referencia ambigua; tabla o alias inválido.
SEM005–009	Tipos incompatibles; cantidad incorrecta de valores; NULL u omisión en columna obligatoria; exceso de VARCHAR; fecha inválida.
SEM010–012	Tabla ya existente; columna duplicada; restricción inválida.
SEM013–015	Índice inexistente; índice duplicado; subconsulta IN o escalar con una cantidad de columnas distinta de una.
SEM016–017	Uso inválido de agregación o columna no agrupada; intento de eliminar una tabla referenciada.
SEM018–020	Literal fuera de rango; condición no booleana; posición de ORDER BY fuera del resultado.

Tabla 7: Agrupación de los veinte códigos semánticos implementados.

Los errores de ejecución son `EXE001` (PRIMARY KEY), `EXE002` (UNIQUE), `EXE003` (FK sin padre), `EXE004` (registro referenciado), `EXE005` (subconsulta escalar con varias filas), `EXE006` (división entre cero), `EXE007` (valor o resultado fuera del tipo admitido) y `EXE008` (almacenamiento o bloqueo).

Una subconsulta escalar con más de una columna se rechaza durante la semántica. Su cantidad de filas se comprueba al evaluarla, porque depende del contenido de la base y de sus predicados. Si retorna cero filas, su resultado escalar es NULL.

7 Diseño del almacenamiento en archivos

7.1 Distribución de los archivos

```
database/
  metadata.txt
  estudiantes.csv
  estudiantes.rows
  cursos.csv
  cursos.rows
  indexes/
    pk_estudiantes.idx
    idx_estudiantes_edad.idx
  .lock
  .transaction/           (solo durante una confirmacion)
```

Todos los datos de la base son archivos de texto. `metadata.txt` describe los esquemas; cada tabla tiene un CSV; cada índice, un archivo `.idx`; y `.rows` relaciona los identificadores de fila con posiciones físicas en el CSV. El bloqueo y el diario son mecanismos de coordinación del almacenamiento, no un servicio de base de datos externo.

7.2 Catálogo: esquemas, tipos y restricciones

El catálogo utiliza instrucciones de texto por línea. Para cada tabla conserva el siguiente rowid libre y sus columnas con tipos y restricciones. Los índices se describen con su tabla, columna y propiedades.

```
TABLE estudiantes
  NEXT_ROWID 9
  COLUMN id INT PRIMARY_KEY NOT_NULL
  COLUMN nombre VARCHAR(50) NOT_NULL
  COLUMN edad INT
  COLUMN promedio FLOAT
  COLUMN activo BOOLEAN
  COLUMN ingreso DATE
  COLUMN carrera_id INT REFERENCES carreras.id
END

INDEX pk_estudiantes ON estudiantes.id UNIQUE AUTO
INDEX idx_estudiantes_edad ON estudiantes.edad
```

La marca AUTO identifica los índices creados por PRIMARY KEY o UNIQUE. No se permite eliminarlos mediante DROP INDEX, porque sostienen la restricción que los originó. El catálogo se lee al abrir la base y se serializa dentro de la misma confirmación que sus archivos afectados.

7.3 Registros CSV y valores especiales

Cada registro comienza con `__rowid`, un identificador interno estable que no cambia cuando se eliminan otras filas. Los índices apuntan a este identificador y no al ordinal de la fila dentro del archivo.

```
__rowid,id,nombre,edad,promedio,activo,ingreso,carrera_id
1,1,"Ana López",20,91.5,TRUE,2023-01-15,1
4,4,"Pedro Méndez",18,,TRUE,2024-01-10,
```

El fragmento anterior ilustra el formato. Todo texto se escribe entre comillas dobles y las comillas internas se duplican. Un campo vacío sin comillas representa NULL y `""` representa una cadena vacía. El lector respeta comas y saltos de línea dentro de campos entrecomillados.

FLOAT se escribe mediante una representación decimal suficientemente precisa para reconstruir el mismo `double`. La conversión de entrada controla el rango y rechaza valores no finitos. VARCHAR cuenta puntos de código de UTF-8: por ejemplo, `José` ocupa cuatro caracteres y puede almacenarse en VARCHAR(4).

7.4 Posiciones de fila y lectura directa

Un índice entrega rowids. Para evitar recorrer el CSV hasta encontrarlos, el archivo `<tabla>.rows` conserva la posición y longitud de cada registro en bytes:

```
ROWS 1 <tamano_del_csv_en_bytes>
<rowid> <offset_en_bytes> <longitud_en_bytes>
```

`readRows` abre el archivo en modo binario, hace `seekg` a cada posición candidata y decodifica únicamente ese registro. Las posiciones incluyen la representación CSV completa, sus comillas y los saltos de línea internos; por eso se calculan al serializar la tabla. INSERT, UPDATE y DELETE confirman el CSV y sus nuevas posiciones juntos.

Al abrir un directorio de posiciones se comprueban su versión, el tamaño del CSV y los límites de sus entradas. Una base antigua sin `.rows` sigue funcionando: se carga la tabla y se calculan las posiciones en memoria; la siguiente modificación las guarda en disco.

7.5 Confirmación, recuperación y concurrencia

Las modificaciones trabajan sobre copias de la tabla y sus índices. Superadas las restricciones, una sentencia prepara un conjunto de escrituras y eliminaciones. `FileTransaction` realiza estos pasos:

1. Registra en un manifiesto todos los archivos afectados y si ya existían.
2. Respalda los originales y termina de escribir todos los temporales. Se comprueba también el cierre de cada archivo.
3. Marca el diario como preparado y comienza a instalar los archivos.
4. Después de instalar todos los cambios, coloca el marcador de confirmación mediante un renombrado.

5. Elimina los temporales y respaldos que ya no se necesitan.

Si falla la preparación, ningún original se modifica. Si falla o se interrumpe la instalación, el diario permite restaurar los respaldos y eliminar archivos nuevos no confirmados. Al volver a abrir la base se recupera una operación pendiente antes de cargar el catálogo. Un diario marcado como confirmado solo necesita limpieza; no se revierte.

Un bloqueo del sistema operativo protege la ejecución completa del CLI. Mientras otro proceso conserva el bloqueo, una nueva ejecución recibe `EXE008`; así no puede recuperar un diario que todavía está activo. El bloqueo se libera al cerrar el proceso, incluso tras una terminación inesperada.

La unidad de confirmación es una sentencia, no el script completo. No se implementan transacciones SQL de varias sentencias ni durabilidad frente a cortes eléctricos mediante `fsync`. La recuperación descrita cubre errores de escritura e interrupciones del proceso.

8 Interpretación y ejecución de consultas

8.1 DDL, INSERT, UPDATE y DELETE

CREATE TABLE genera el esquema, el CSV vacío, las posiciones y los índices automáticos. DROP TABLE elimina la entrada del catálogo y los archivos asociados, siempre que ninguna otra tabla la referencie. CREATE INDEX construye un índice a partir de los datos existentes; si es UNIQUE, rechaza claves repetidas antes de confirmarlo.

INSERT evalúa cada fila de VALUES, convierte sus valores y comprueba NOT NULL, unicidad y claves foráneas. Las columnas omitidas reciben NULL cuando el esquema lo permite. Si cualquier fila falla, la sentencia entera se descarta.

UPDATE localiza sus candidatos, evalúa todas las asignaciones sobre la fila original y valida los resultados. Retira primero las claves anteriores de los índices y luego incorpora las nuevas. Esto permite, por ejemplo, desplazar varias claves sin detectar duplicados transitorios que no existirán en el estado final.

DELETE evalúa WHERE y verifica las referencias externas antes de retirar filas y entradas de índices. Los cambios se confirman junto con las posiciones físicas correspondientes.

8.2 Orden de evaluación de SELECT

El orden lógico implementado es el siguiente:

```
FROM -> INNER JOIN -> WHERE
     -> GROUP BY / agregaciones -> HAVING
     -> proyeccion SELECT -> DISTINCT -> ORDER BY -> LIMIT
```

WHERE se evalúa sobre cada fila candidata, incluso cuando se utilizó un índice para reducir el conjunto. La proyección produce las expresiones solicitadas y expande `*` o `tabla.*` a columnas concretas. ORDER BY admite expresiones, alias de salida y posiciones válidas.

8.3 JOIN, agrupación y agregaciones

INNER JOIN combina las filas que cumplen ON. Si una columna indexada de la tabla derecha se compara con un valor ya conocido de la izquierda, se emplea un *Index Nested Loop*; en otro

caso se prueban las combinaciones mediante *Nested Loop*. Cada unión amplía la fila combinada y conserva las posiciones resueltas durante la semántica.

GROUP BY construye grupos por el valor de sus expresiones. COUNT(*) cuenta filas; COUNT(expr) cuenta valores no nulos. SUM y AVG exigen argumentos numéricos; MIN y MAX seleccionan extremos de tipos admitidos. Las agregaciones ignoran NULL en sus argumentos. Sin filas, COUNT produce 0 y las demás agregaciones producen NULL.

En una consulta agregada, las columnas fuera de funciones deben estar agrupadas. HAVING filtra grupos después de las agregaciones. Su presencia sin GROUP BY crea un grupo implícito, incluso sin funciones agregadas: `SELECT 1 HAVING FALSE;` devuelve cero filas.

8.4 Subconsultas y lógica de NULL

IN y NOT IN comparan una expresión con una lista o con la columna de una subconsulta. EXISTS y NOT EXISTS verifican si esta produce filas. Una subconsulta escalar debe tener una columna y, al evaluarse, como máximo una fila. Estos conceptos se contrastaron con documentación SQL pública como referencia semántica; esa consulta documental no incorpora ningún DBMS al programa [5].

Las subconsultas no correlacionadas se calculan una vez por sentencia y se guardan en caché. Las correlacionadas usan las columnas externas y se reevalúan para cada contexto externo necesario. El plan registra la correlación, la cantidad de ejecuciones y el plan de la primera evaluación.

Si IN no encuentra coincidencia y existe NULL en el conjunto, el resultado es desconocido. El conjunto vacío tiene un tratamiento distinto: `NULL IN (subconsulta vacía)` es FALSE y su negación es TRUE. Esta distinción evita perder filas con `NOT IN` cuando la subconsulta no produce resultados.

9 Índices y planificación del acceso

Cada índice se representa en memoria mediante un `std::map<Value, std::set<rowid»`. Es una estructura ordenada con búsquedas logarítmicas sobre sus claves; no es un árbol B almacenado por páginas. El archivo `.idx` es texto ordenado y se carga cuando se necesita.

```
# INDEX idx_estudiantes_edad ON estudiantes(edad)
# valor,rowids (ordenado por valor)
16,8
17,2
18,4
19,5
20,1
21,7
22,3
25,6
```

Las claves NULL se omiten. Los comentarios pertenecen únicamente al encabezado: una línea que comienza con `#` dentro de una cadena multilínea forma parte de la clave y debe conservarse.

El planificador reconoce igualdades, IN no negado y comparaciones de rango, además de `<>`, cuando el otro operando puede evaluarse antes de leer la tabla objetivo. También aprovecha valores de tablas anteriores en un JOIN o de consultas externas.

Predicado	Acceso
`edad >= 18`	Busca las claves del rango y obtiene sus rowids.
`id IN (1,3)`	Une los rowids encontrados para cada valor.
`id = 1 OR id = 3`	Une ambos accesos si las dos ramas pueden usar índices.
`id >= 2 AND edad >= 19`	Intersecta los accesos disponibles; evalúa el predicado completo al final.
OR con una rama no indexable	Recorre la tabla para no excluir resultados de esa rama.

Tabla 8: Decisiones del planificador para predicados indexables.

INSERT agrega claves, UPDATE retira las anteriores e incorpora las nuevas y DELETE elimina sus referencias. El archivo de posiciones permite traducir los rowids a lecturas directas del CSV. El mismo mecanismo se utiliza en los JOIN indexados.

Evidencia de lectura selectiva

La prueba de una consulta fría por PRIMARY KEY sobre estudiantes verifica que el almacenamiento decodifica una fila. Otra prueba combina tres tablas con filtros e índices y verifica tres filas leídas en total. No se comprueba únicamente la presencia de una etiqueta en el plan.

El contador `database.reads.table_rows` registra filas decodificadas por el almacenamiento. No equivale a tiempo de ejecución ni al total de bytes del sistema operativo: también se cargan las estructuras de índices y posiciones, y el explorador del IDE lee una vista previa acotada de los archivos. Las modificaciones continúan reescribiendo la tabla completa.

10 Interfaz gráfica y evidencia de funcionamiento

El servidor Flask ejecuta `sql_cli` con la carpeta de base de datos y el SQL recibido; devuelve al navegador el JSON del núcleo. Las rutas `/api/run`, `/api/database` y `/api/tests` conectan ejecución, inspección y pruebas. La separación entre rutas, plantillas y archivos estáticos sigue la organización habitual de Flask [4].

Las capturas siguientes proceden de la aplicación real, ejecutada con una base de demostración aislada. Se cargaron los ejemplos 01 y 02 y se crearon los índices de edad y estudiante de inscripción. Los datos contienen 3 carreras, 8 estudiantes, 6 cursos y 13 inscripciones. Las imágenes muestran detalles de los paneles, con zoom para facilitar su lectura; no son maquetas.

10.1 Consulta filtrada y plan de ejecución

La consulta del enunciado se amplió para mostrar edad y ordenar el resultado. Devuelve seis estudiantes: Pedro (18), María (19), Ana (20), Lucía (21), Sofía (22) y Jorge (25). El plan utiliza `idx_estudiantes_edad` y el contador registra seis filas decodificadas de las ocho existentes.

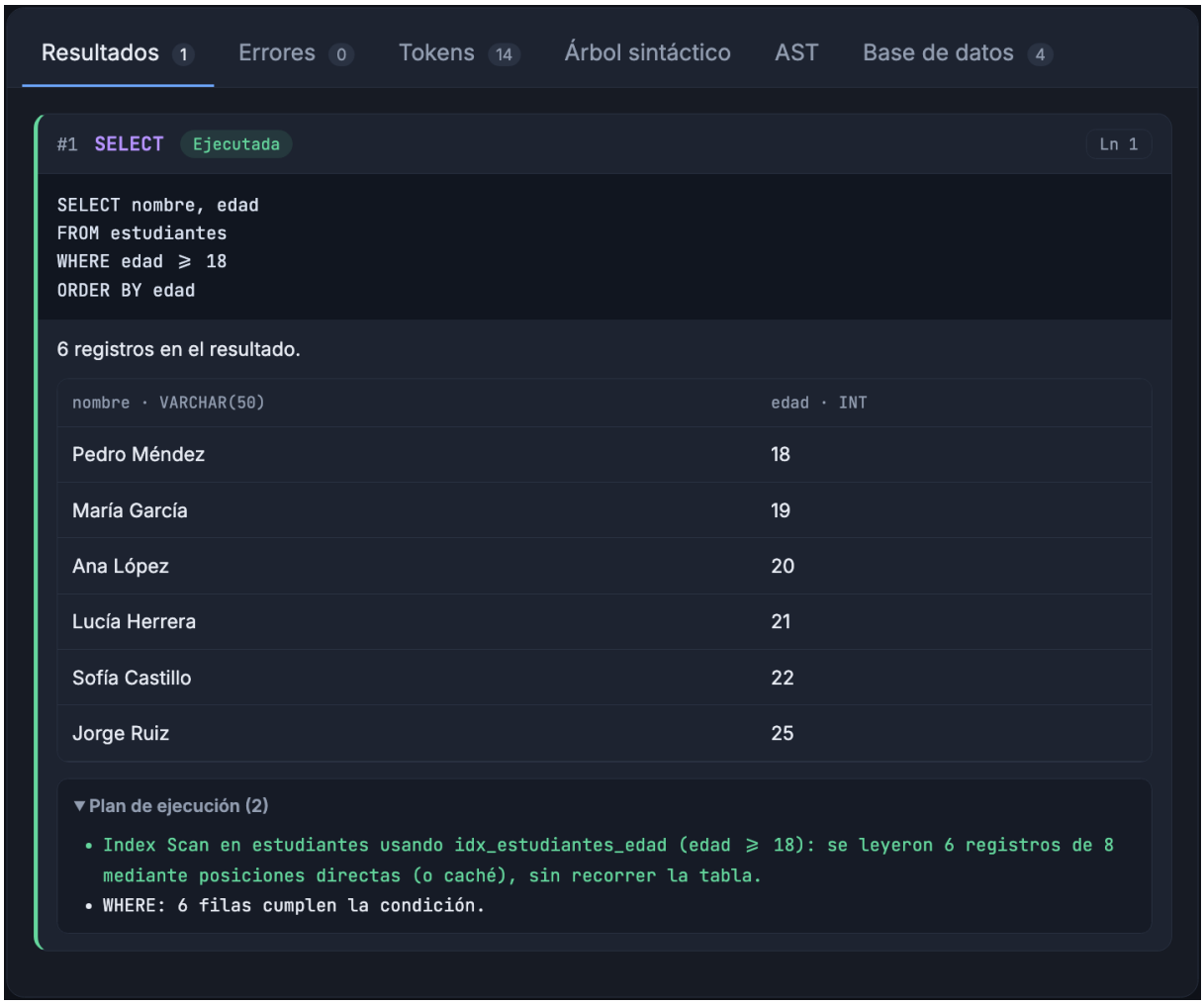


Figura 1: SELECT con WHERE, ORDER BY, resultados y acceso por índice de edad.

10.2 Tokens

La tabla de tokens expone categoría, tipo ANTLR, texto y ubicación. Permite verificar que una palabra reservada, un identificador o un operador llegaron correctamente al parser.



#	CATEGORÍA	TOKEN ANTLR	LEXEMA	LÍNEA	COLUMNA
1	Palabra reservada	K_SELECT	SELECT	1	1
2	Identificador	IDENTIFIER	nombre	1	8
3	Palabra reservada	K_FROM	FROM	2	1
4	Identificador	IDENTIFIER	estudiantes	2	6
5	Palabra reservada	K_WHERE	WHERE	3	1
6	Identificador	IDENTIFIER	edad	3	7
7	Operador de comparación	GTE	⩾	3	12
8	Literal entero	INT_LITERAL	18	3	15
9	Fin de sentencia	SEMI	;	3	17

Figura 2: Tokens de la consulta SELECT nombre FROM estudiantes WHERE edad >= 18.

10.3 Árbol sintáctico concreto

El árbol sintáctico conserva las reglas de la gramática y sus hojas. El visor permite expandir, contraer y cambiar el zoom para recorrer la estructura reconocida por ANTLR.

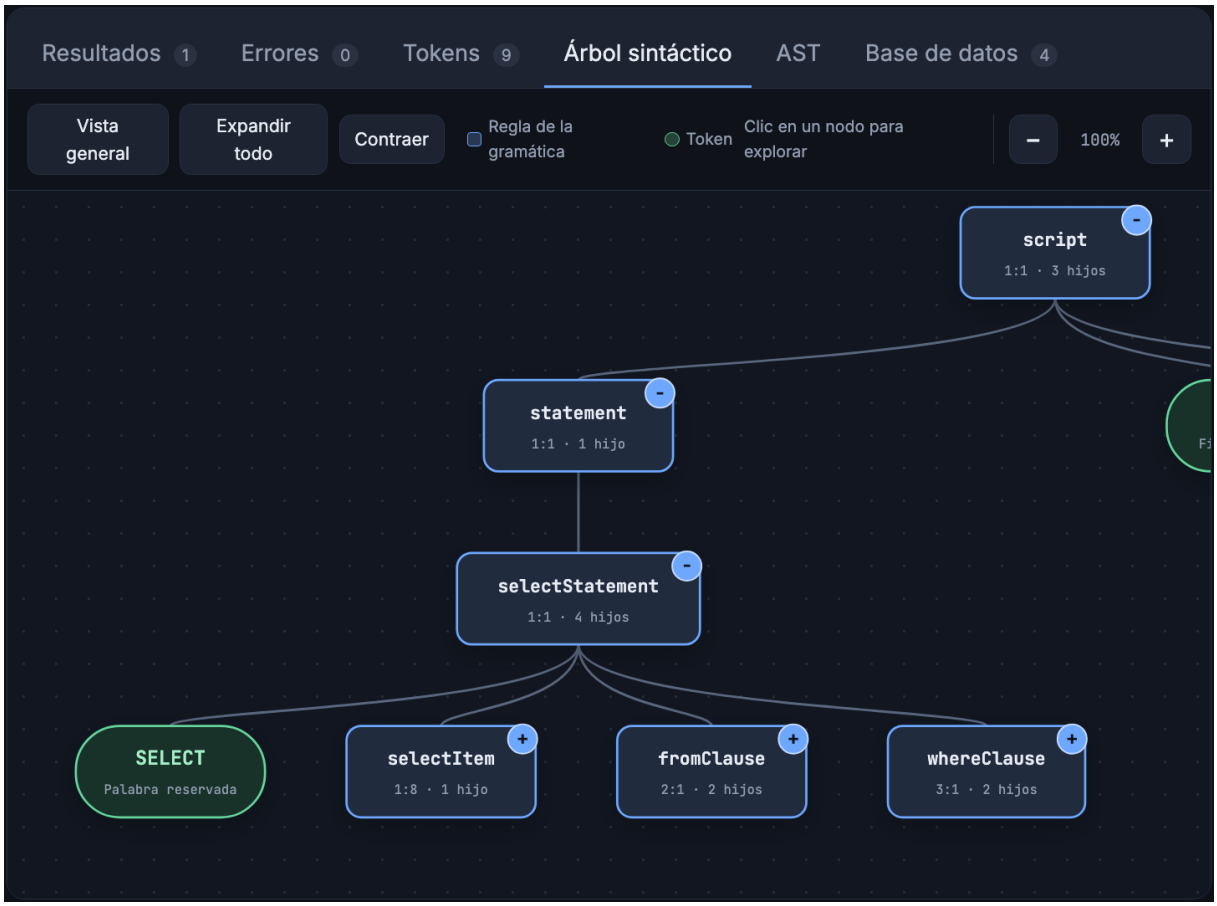


Figura 3: Árbol sintáctico concreto de una consulta, en modo de análisis.

10.4 AST visible y tipos resueltos

El AST representa la misma entrada mediante SELECT, proyección, FROM y WHERE. Las columnas ya muestran sus tipos. Esta vista permite relacionar la consulta escrita con la estructura utilizada por la semántica y el intérprete, sin los niveles auxiliares del árbol concreto.

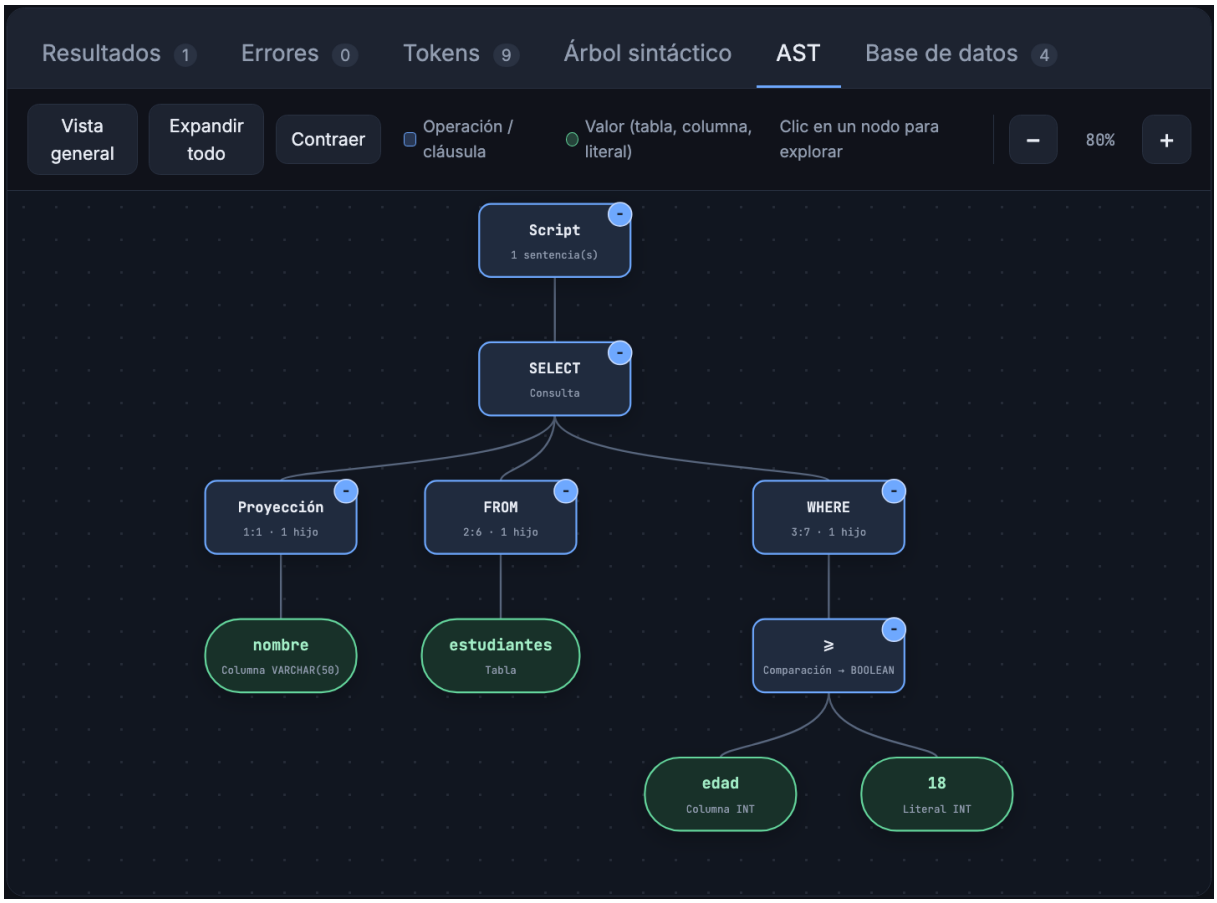


Figura 4: AST decorado con la proyección, la tabla y el predicado de comparación.

10.5 JOIN, agregaciones y HAVING

La consulta une cursos e inscripciones, cuenta las inscripciones por curso y calcula la nota máxima. HAVING conserva los cursos con al menos dos inscripciones; ORDER BY utiliza el alias del conteo. Se obtienen cinco grupos. Cálculo tiene cuatro inscripciones y Compiladores tres.

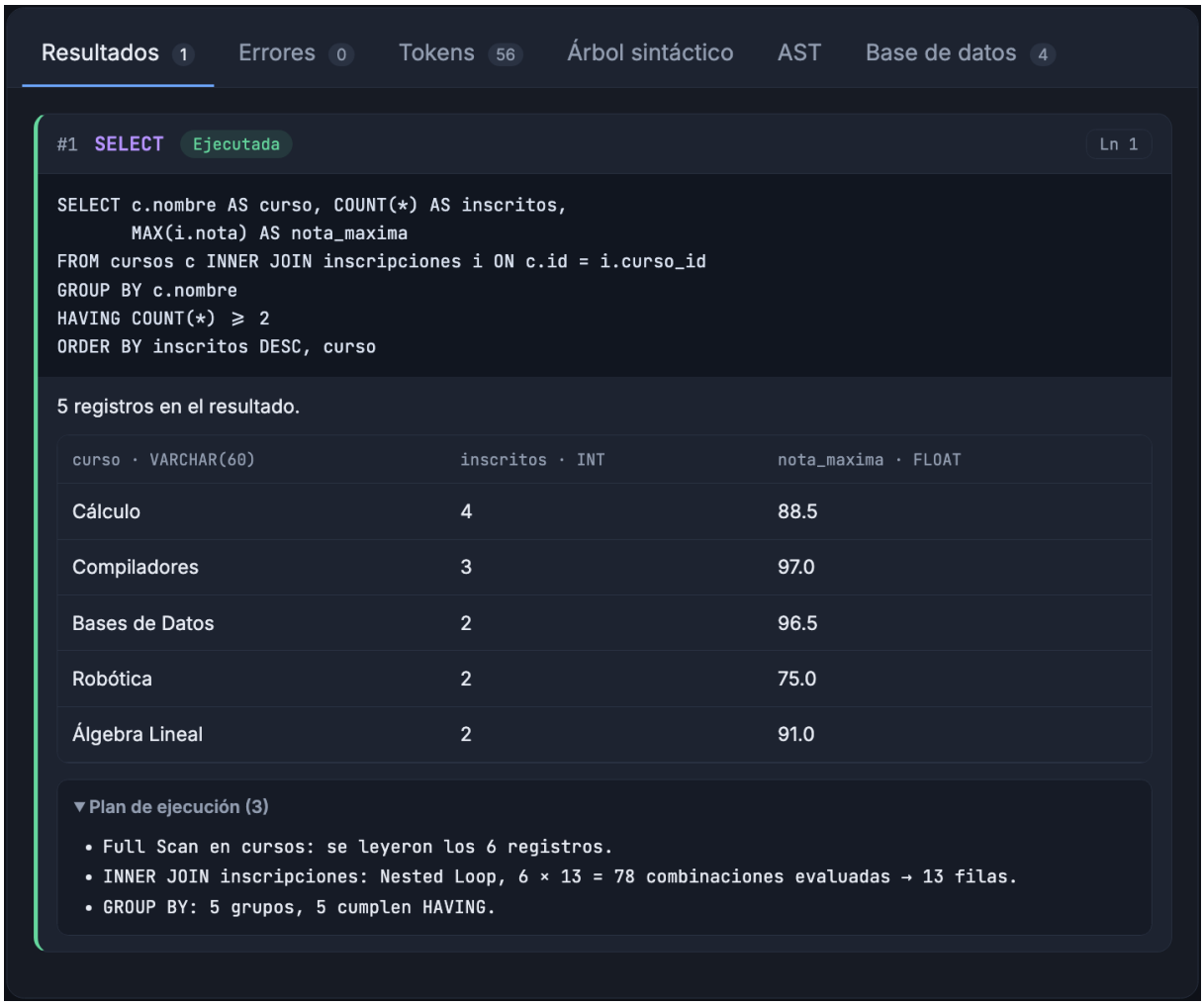


Figura 5: INNER JOIN con COUNT, MAX, GROUP BY, HAVING y ORDER BY.

10.6 Subconsultas con IN

El ejemplo con IN retorna a Ana López, Luis Pérez, María García y Sofía Castillo: sus identificadores aparecen entre las inscripciones del curso 3. La subconsulta se calcula una vez y sus valores se buscan en el índice de la PRIMARY KEY de estudiantes.

```
SELECT nombre
FROM estudiantes
WHERE id IN (SELECT estudiante_id
             FROM inscripciones WHERE curso_id = 3)
ORDER BY nombre;
```

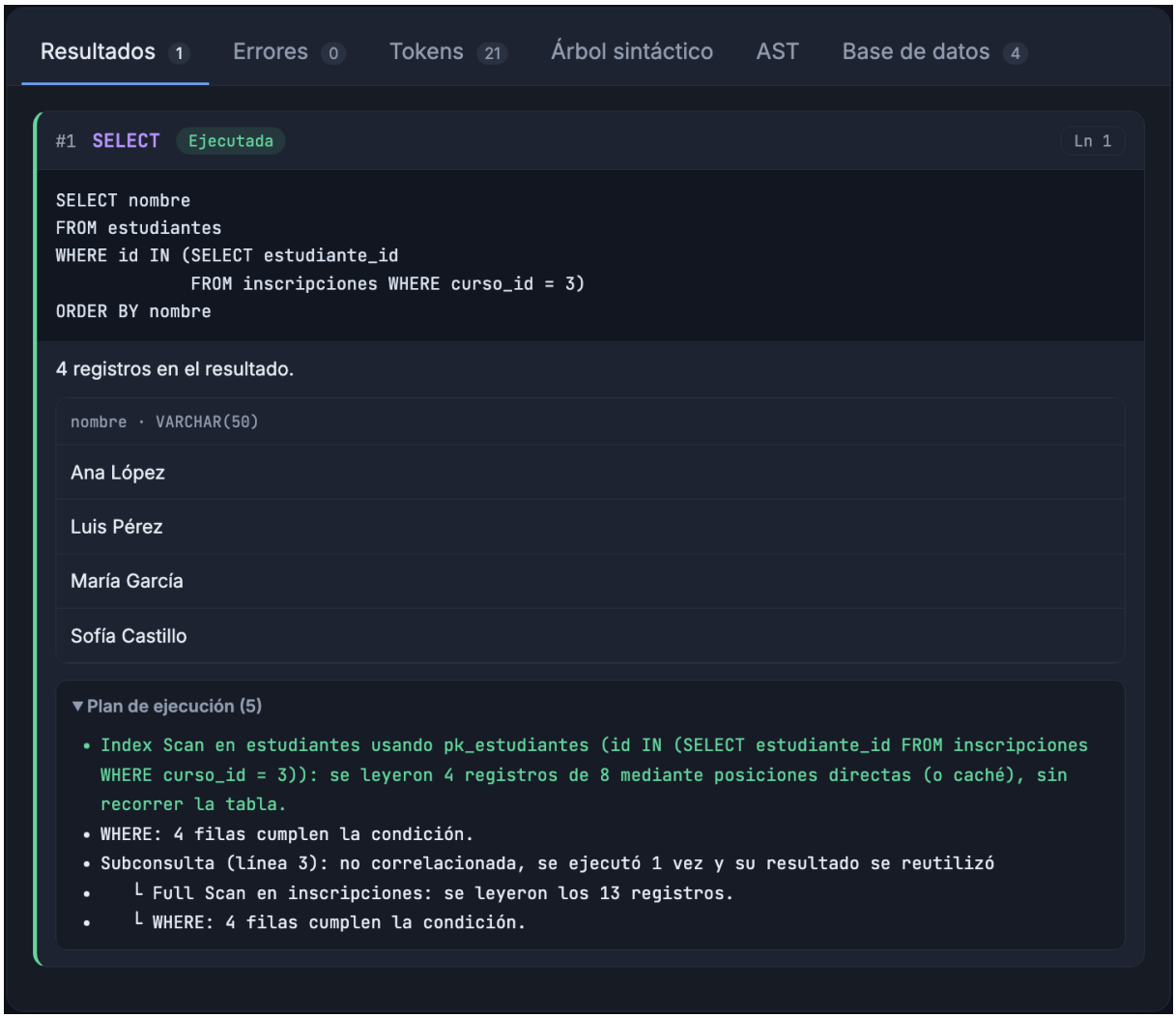


Figura 6: IN con SELECT anidado y acceso al índice de estudiantes.

10.7 EXISTS correlacionado y subconsulta escalar

En el caso correlacionado se buscan estudiantes con alguna nota de al menos 95. Solo Lucía Herrera cumple. La subconsulta usa `e.id` del contexto externo y el plan registra ocho evaluaciones, una por estudiante, con búsquedas por el índice de inscripciones.

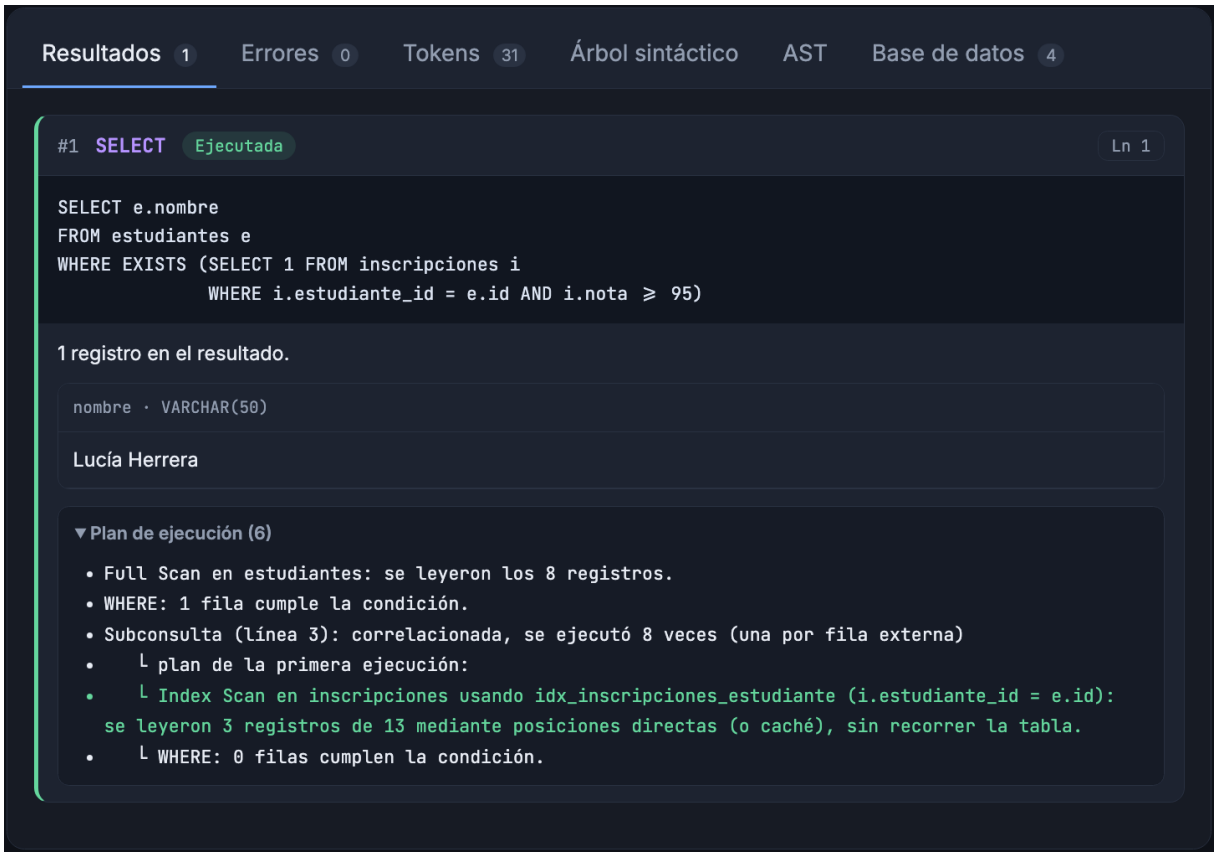


Figura 7: EXISTS correlacionado con una referencia a la fila externa.

La variante escalar compara el promedio individual con el promedio global de 82.75. Devuelve Lucía (95), Ana (91.5), María (88) y Sofía (85.25). El código y el resultado reproducible de esta variante se incluyen en `evidencia/sql/escalar.sql` y `evidencia/consultas-resultados.json`.

10.8 Catálogo e índices persistidos

La pestaña Base de datos muestra las columnas, restricciones, cantidad de filas e índices de cada tabla, junto con el contenido de los archivos. El archivo elegido en la figura siguiente es el índice de edad: sus claves y rowids corresponden al ejemplo descrito en el diseño.

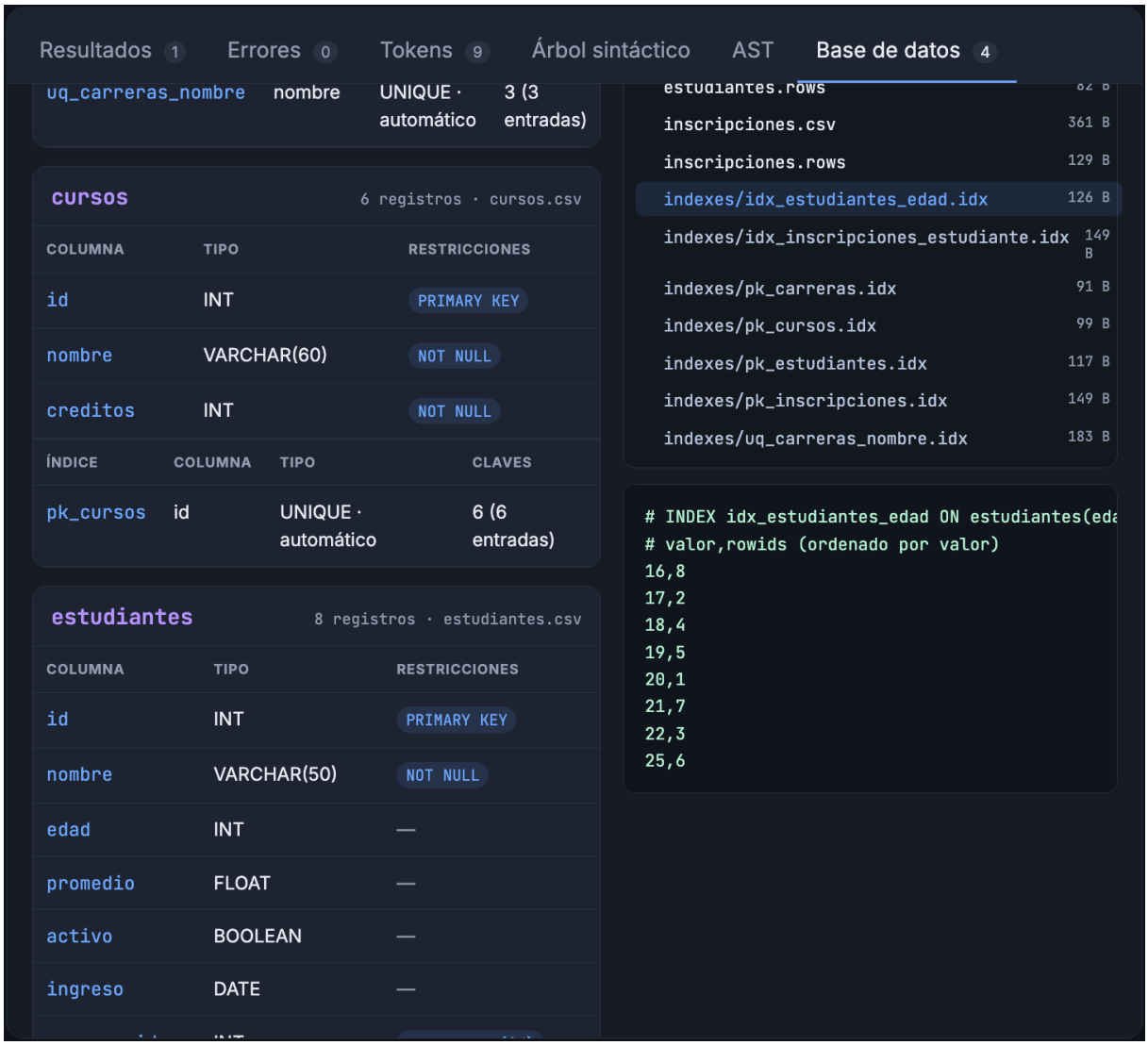


Figura 8: Explorador del almacenamiento con el contenido real de un índice .idx.

10.9 Rechazo de entradas inválidas

La primera captura reúne tres errores estáticos independientes: una tabla inexistente, una columna inexistente y un valor de INSERT incompatible. El script no se ejecuta. La segunda muestra un error que depende de la evaluación: división entre cero, reportada como `EXE006`.

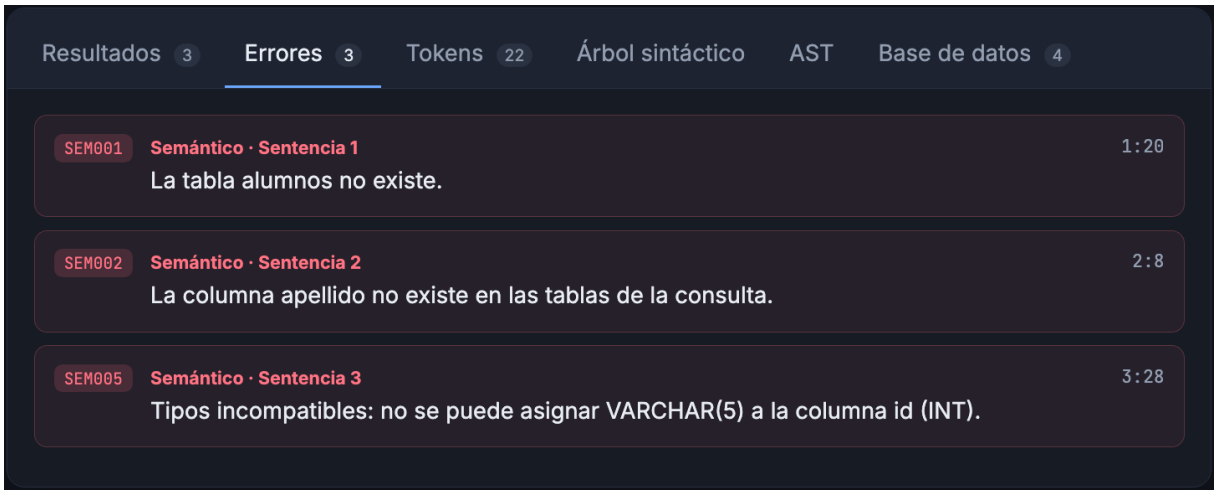


Figura 9: Errores semánticos con código, sentencia y ubicación en el editor.

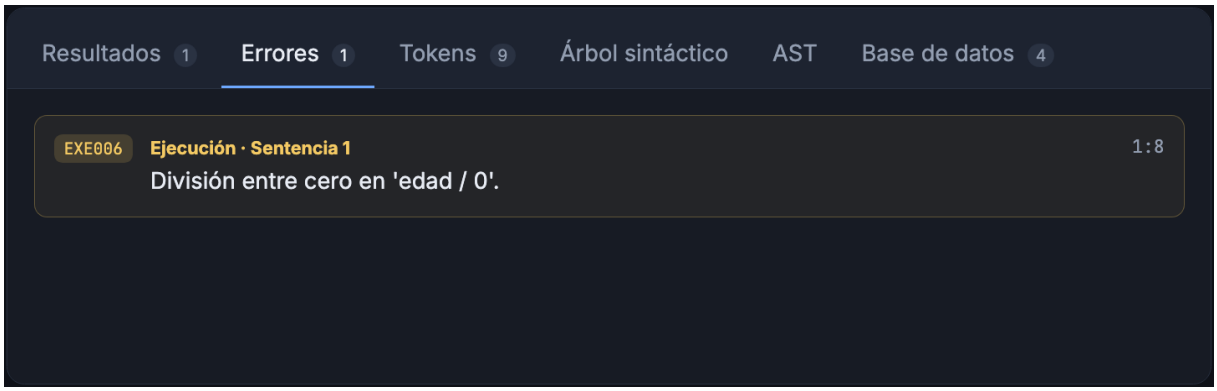


Figura 10: Error de ejecución por división entre cero.

11 Pruebas y resultados de validación

11.1 Metodología

`sql_tests` invoca el mismo servicio C++ que utiliza el IDE. Cada caso tiene una carpeta temporal aislada; cuando corresponde, se carga un esquema de ejemplo. La verificación compara filas, tipos, diagnósticos, planes, contadores y contenido de archivos. Algunos casos realizan una segunda ejecución para comprobar persistencia.

Un error esperado aprueba cuando el intérprete rechaza la entrada con el código requerido. El total de 76 casos válidos corresponde a la clasificación del reporte: incluye también verificaciones de omisión de fases y de recuperación que examinan respuestas fallidas deliberadas. Los 41 errores esperados no son fallos del sistema.

Categoría	**Casos**	**Aprobados**
Lexer y parser	7	7
AST	4	4
Análisis semántico	26	26
CREATE, DROP e INSERT	10	10
SELECT, WHERE, UPDATE y DELETE	12	12
JOIN, agregaciones y GROUP BY	7	7
Índices	10	10
Subconsultas	7	7
Almacenamiento en archivos	3	3
Regresiones de robustez	31	31
Total	**117**	**117**

Tabla 9: Resultados de la batería ejecutada para esta versión del informe.

11.2 Regresiones incorporadas

Las 31 regresiones adicionales se organizan en dos conjuntos: 16 casos de tipos, expresiones y texto indexado; y 15 de planificación, almacenamiento, recuperación y concurrencia. Entre las verificaciones principales se encuentran las siguientes:

- VARCHAR con acentos y caracteres de varios bytes, tanto en validación semántica como en valores calculados por UPDATE.
- Literales FLOAT fuera de rango, valores subnormales, precisión después de reabrir y desbordamientos de aritmética y SUM.
- HAVING con grupo implícito y NULL frente a subconsultas vacías.
- Claves indexadas con saltos de línea, líneas vacías y `#`.
- Unión e intersección de índices y recorrido correcto de un OR con una rama no indexable.
- Filas efectivamente decodificadas y actualización de posiciones tras INSERT, UPDATE y DELETE.
- Errores de escritura durante INSERT, UPDATE, CREATE y DROP sin confirmar cambios parciales.

- Recuperación de una instalación interrumpida y conservación de una confirmación terminada cuyo diario requiere limpieza.
- Rechazo de una segunda ejecución sobre una base bloqueada.

También se ejecutó la batería con AddressSanitizer y UndefinedBehaviorSanitizer: los 117 casos aprobaron sin diagnósticos de esas herramientas. Esta comprobación complementa las pruebas funcionales; no constituye una demostración de ausencia de errores en entradas no cubiertas.

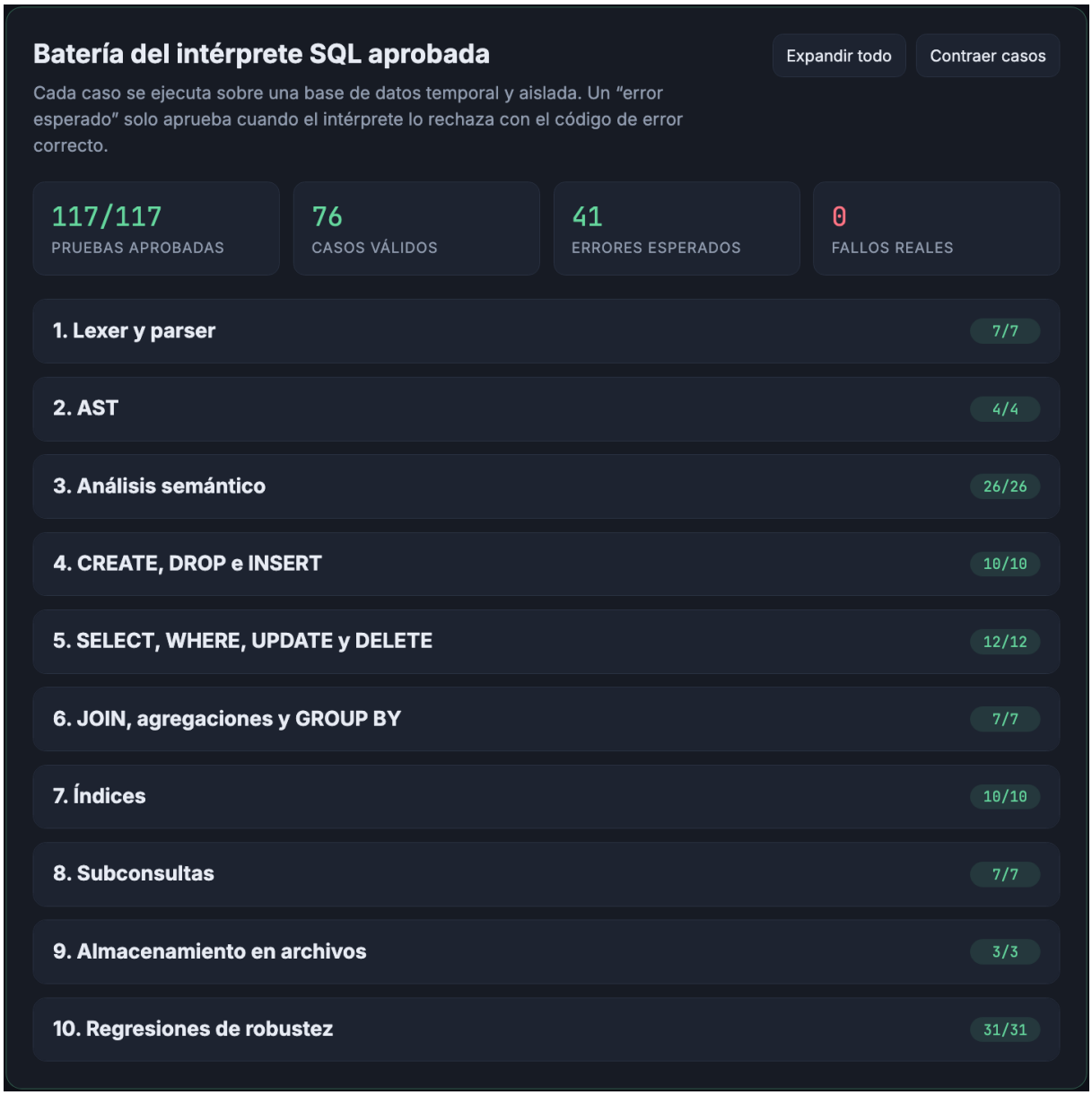


Figura 11: Reporte de la batería del intérprete SQL: 117 aprobadas y cero fallos reales.

11.3 Evidencia reproducible

La carpeta del informe conserva `evidencia/pruebas.json`, con el reporte íntegro, y `evidencia/consultas-resultados.json`, con SQL, resultados, diagnósticos, planes y contadores de las demostraciones. `evidencia/sql/00_preparar_base.sql` reproduce el esquema, los datos y los

índices usados en las capturas. Cada consulta dispone de un archivo SQL independiente para que las demostraciones de error no impidan ejecutar las consultas válidas.

12 Trazabilidad del desarrollo

La rama del proyecto contiene seis commits de implementación inicial y seis de refuerzo. En conjunto, los doce commits descritos tienen una distribución de seis por integrante. Las identidades coinciden con las utilizadas en las entregas anteriores.

Usuario	Correo	Integrante
G1LB3T0	luis.yo09@hotmail.com	Luis Gonzalez, 23353
jaq23369	jaq23369@uvg.edu.gt	Joel Jaquez, 23369

Tabla 10: Identidades registradas en Git.

Commit	Autor	Aporte
68ae80c	G1LB3T0	Gramática SQL y generación del parser ANTLR.
0426696	jaq23369	Construcción del AST y análisis semántico SQL.
2016eb1	G1LB3T0	Almacenamiento en archivos, intérprete e integración del CLI.
f0ad7c3	G1LB3T0	Batería inicial de 86 pruebas de integración.
ef55839	jaq23369	Adaptación del IDE, endpoints y ejemplos SQL.
510c384	jaq23369	Guía de arquitectura, reutilización y defensa.
2f7be06	G1LB3T0	Unicode, tipos numéricos, HAVING y condiciones con NULL.
bdd28f8	jaq23369	Confirmación conjunta, recuperación y bloqueo de archivos.
10056ad	G1LB3T0	Acceso por posiciones y combinación de índices.
23b9d6b	jaq23369	Dieciséis regresiones de tipos y expresiones.
c0a4b01	G1LB3T0	Quince regresiones de acceso, disco y recuperación; 117 casos.
037ac10	jaq23369	Documentación del almacenamiento y de su validación.

Tabla 11: Historial funcional del proyecto SQL, en orden cronológico.

13 Compilación y ejecución

Se requiere un compilador C++17, CMake 3.20 o superior, Java para generar ANTLR y Python con Flask para el IDE. Los archivos C++ generados ya están en el repositorio; los scripts de construcción también permiten regenerarlos desde la gramática.

```
# macOS o Linux
./scripts/build_sql.sh

# Windows (PowerShell)
.\scripts\build_sql.ps1

# Pruebas de integracion
ctest --test-dir build/sql --output-on-failure

# Ejecucion desde consola
./sql_cli --db database --file ejemplo.sql
./sql_cli --db database --analyze --file ejemplo.sql
./sql_cli --db database --catalog

# IDE
python3 app.py
# Abrir http://127.0.0.1:5050
```

Si el runtime de ANTLR ya se encuentra descargado, la variable `ANTLR4_LOCAL_SOURCE` permite construir sin volver a descargarlo. El CLI devuelve un documento JSON y utiliza código de salida 0 para éxito, 1 para errores del pipeline y 2 para problemas de uso o lectura de la entrada.

Para reproducir las figuras en otra base de datos se recomienda utilizar una carpeta nueva y ejecutar primero el script de preparación de `evidencia/sql/`. Los scripts de error deben ejecutarse por separado, puesto que un error semántico impide ejecutar todo su script.

14 Alcance, límites y mejoras futuras

El proyecto implementa un subconjunto SQL orientado a los requisitos del curso. Sus límites deben diferenciarse de los comportamientos comprobados:

- Se implementa INNER JOIN. No se incluyen LEFT JOIN, consultas UNION, vistas ni procedimientos almacenados.
- Los índices, las claves primarias y las claves foráneas son de una columna. No hay un índice B+ por páginas ni un optimizador basado en estimaciones de costo.
- Los índices y las posiciones se cargan en memoria. Las lecturas indexadas del CSV son selectivas; los recorridos completos y las modificaciones cargan la tabla y pueden reescribirla completa.
- La colación de texto es binaria. VARCHAR mide puntos de código, sin normalización Unicode ni conteo de grafemas compuestos.
- Las claves foráneas utilizan RESTRICT; no se implementa CASCADE.
- La confirmación cubre una sentencia. No hay BEGIN/COMMIT SQL de varias sentencias ni garantía frente a cortes eléctricos con fsync.
- El bloqueo exclusivo evita ejecuciones simultáneas sobre la misma carpeta; no ofrece concurrencia de múltiples lectores y escritores.

Como evolución, sería posible incorporar índices compuestos y por páginas, actualización localizada de registros, planificación por costo y un protocolo de durabilidad con sincronización de archivos. Estas mejoras no forman parte de los resultados afirmados por esta entrega.

15 Conclusiones

1. Las técnicas de construcción de compiladores permiten interpretar un lenguaje de consultas completo dentro del alcance del curso: reconocimiento, AST, resolución semántica e interpretación quedan separados y son observables en la interfaz.

2. El almacenamiento propio representa esquemas, tipos, restricciones, filas e índices mediante archivos de texto. El catálogo permite validar el SQL y el diario mantiene la coherencia de las modificaciones ante los fallos cubiertos.

3. Los índices se utilizan para obtener candidatos y las posiciones persistidas permiten leerlos directamente del CSV. La decisión de acceso se comprueba con resultados y contadores de filas, además del plan mostrado al usuario.

4. La reutilización de ANTLR, el contrato JSON, Flask y el visor del IDE permitió concentrar el trabajo nuevo en el lenguaje SQL y su ejecución. El resultado cumple un flujo de interpretación distinto de la generación de código intermedio de Compiscript.

5. Las 117 pruebas y las demostraciones reproducibles aportan evidencia del comportamiento funcional y de los casos límite examinados. La interfaz y el informe permiten seguir una sentencia desde sus tokens hasta los resultados o los archivos modificados.

Referencias

[1] Construcción de Compiladores. *Proyecto 2.1: Intérprete SQL con Almacenamiento en Archivos*. Enunciado del curso, septiembre de 2026.

[2] ANTLR. *ANTLR: herramienta de reconocimiento de lenguajes*. `https://www.antlr.org/`. Consulta: 31 de octubre de 2026.

[3] ANTLR. *The C++ Target*. Documentación del proyecto. `https://github.com/antlr/antlr4/blob/dev/doc/cpp-target.md`. Consulta: 31 de octubre de 2026.

[4] Pallets Projects. *Flask: Quickstart*. `https://flask.palletsprojects.com/en/stable/quickstart/`. Consulta: 31 de octubre de 2026.

[5] PostgreSQL Global Development Group. *Subquery Expressions*. Referencia documental sobre EXISTS, IN y NOT IN. `https://www.postgresql.org/docs/current/functions-subquery.html`. Consulta: 31 de octubre de 2026. No se utiliza PostgreSQL en el sistema.

[6] Gonzalez, L. y Jaquez, J. *Intérprete SQL*, rama `Compiladores2-Proyecto3`, versión `037ac10`. `https://github.com/G1LB3T0/Proyecto1_Compis_Generador_Analizador_Lexico/tree/Compiladores2-Proyecto3`. Fuentes, ejemplos, pruebas y `docs/proyecto3-interprete-sql.md`.